BUSSENSE · HACKTURIN 2026

Il modello che riconosce buche e rifiuti

Rete neurale MobileNetV2 int8 che gira sulla Arduino UNO Q, dentro il POD sul bus

98,0%
accuratezza su foto mai viste

41 ms
per foto sulla UNO Q

2,6 MB
il modello sulla board

0
falsi allarmi dal vivo

Come funziona BusSense

La foto viene analizzata sulla UNO Q: alla piattaforma arriva solo l'esito

TELEFONO

Galaxy A26

Fotocamera
1 foto ogni ~4 m percorsi
1 al secondo in modalità demo

GPS
1 Hz, precisione 5–15 m

Nitidezza
scarta le foto mosse

fa da hotspot Wi-Fi

Wi-Fi
foto + GPS

ARDUINO UNO Q

Dentro il POD, sul bus

Linux · Qualcomm QRB2210 · 4 core Cortex-A53

Riceve
`receiver.py`
foto solo in RAM
mai su disco

Riconosce
`classifier.py`
MobileNetV2 int8
41 ms per foto

Decide
`vision.py`
soglia 0,70
2 foto su 3
no doppioni 20 m

Invia
`cloud.py`
coda su disco
riprova senza rete

Bridge: misure

Microcontrollore STM32U585 · sensori e display

- AHT20 · temp. e umidità
- SGP30 · CO2eq, TVOC
- Luce · A0
- IMU · accelerazioni
- OLED · stato
- Pulsante · registra

HTTPS
chiave API

PIATTAFORMA

TorinoSegnala

Segnalazioni
buca, rifiuti, cassonetto
+ posizione + sensori

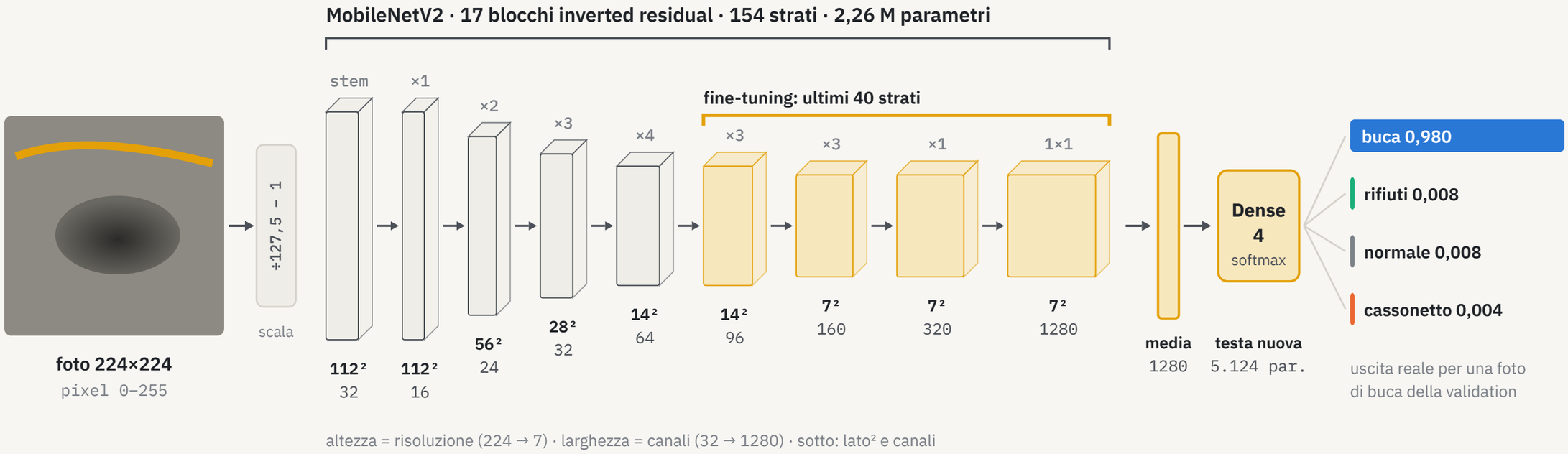
Misure dei sensori
ogni 60 secondi

Mappa
pubblica e area Comune

Le foto non lasciano mai la board. Alla piattaforma arrivano solo categoria, confidenza, posizione e misure dei sensori.

La rete neurale

MobileNetV2 pre-addestrata su ImageNet + una testa nuova a 4 classi



altezza = risoluzione (224 → 7) · larghezza = canali (32 → 1280) · sotto: lato² e canali

2.263.108
parametri

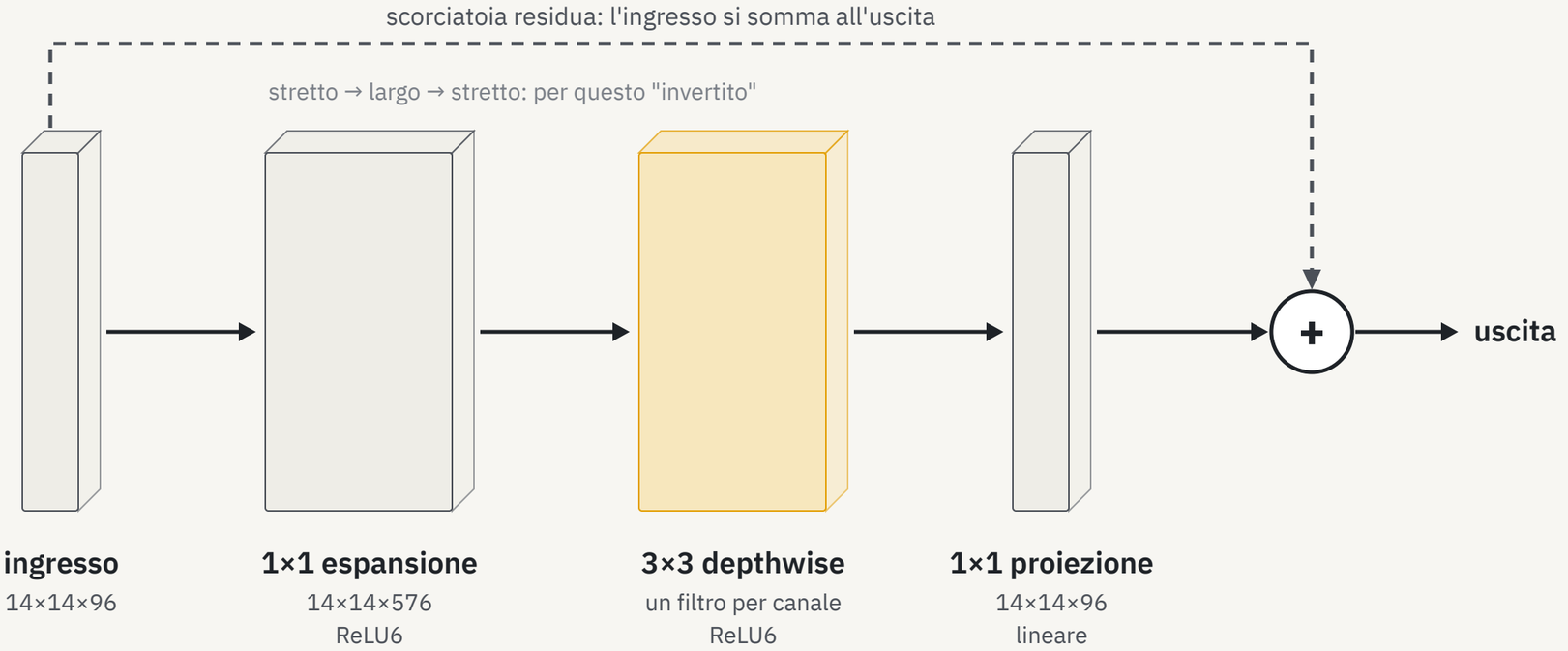
69
operatori int8

2,6 MB
sul disco della board

41 ms
per foto sulla UNO Q

Il mattone della rete: il blocco inverted residual

17 blocchi in fila. Esempio con i numeri reali di un blocco a 14×14 e 96 canali



Perché MobileNetV2

Le convoluzioni depthwise separabili costano ~8 volte meno: la rete gira sui 4 core della UNO Q in 41 ms.

Perché ReLU6

Limita i valori fra 0 e 6: stanno bene in 8 bit, per questo la versione int8 non perde accuratezza (98,0%).

Moltiplicazioni per uno strato 3×3 da 96 a 96 canali su 14×14



8,2×
meno calcoli

Come è stato addestrato

Dal dataset al file che gira sulla board: sei passi, pochi minuti su una GPU da portatile



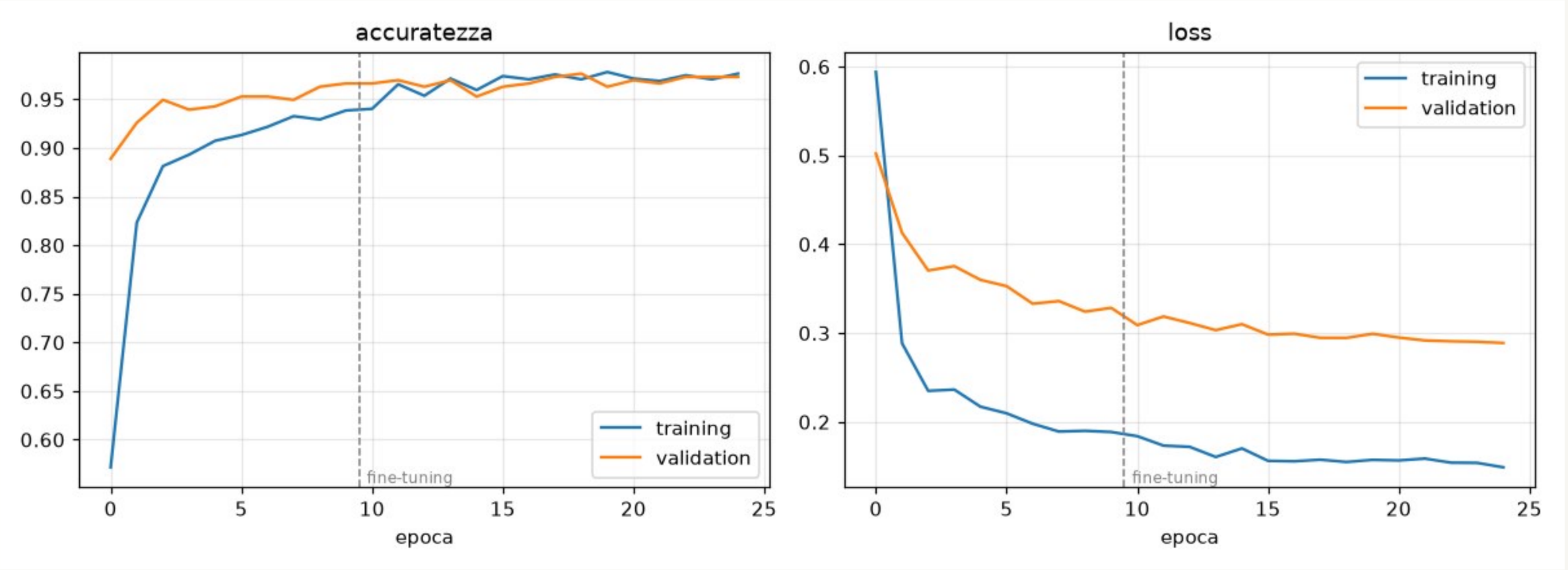
98,0%
accuratezza sulle 297 foto di validation

+0,3
punti dopo la quantizzazione int8: nessun costo

0
falsi allarmi sulle foto di strada normale

Le curve di addestramento

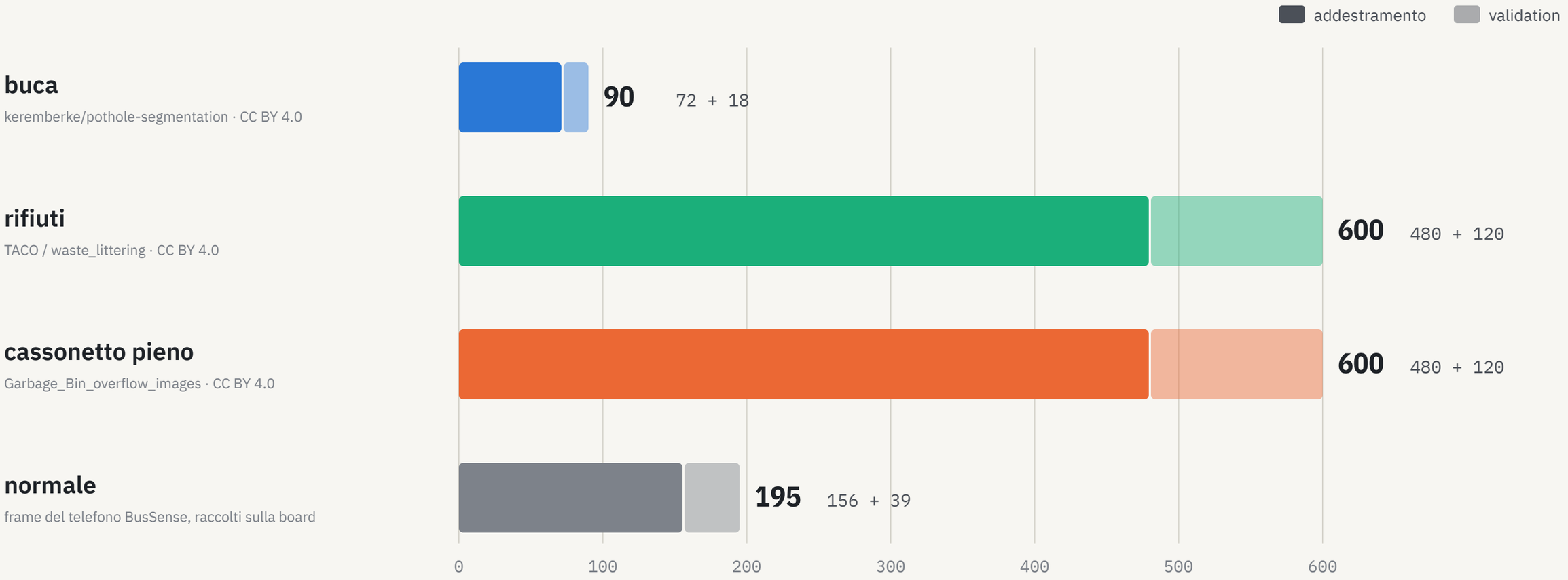
Training e validation salgono insieme: nessun overfitting



10 epoche solo testa, poi 15 di fine-tuning (riga tratteggiata)

Il dataset

1.485 immagini in 4 classi · 80% addestramento, 20% validation

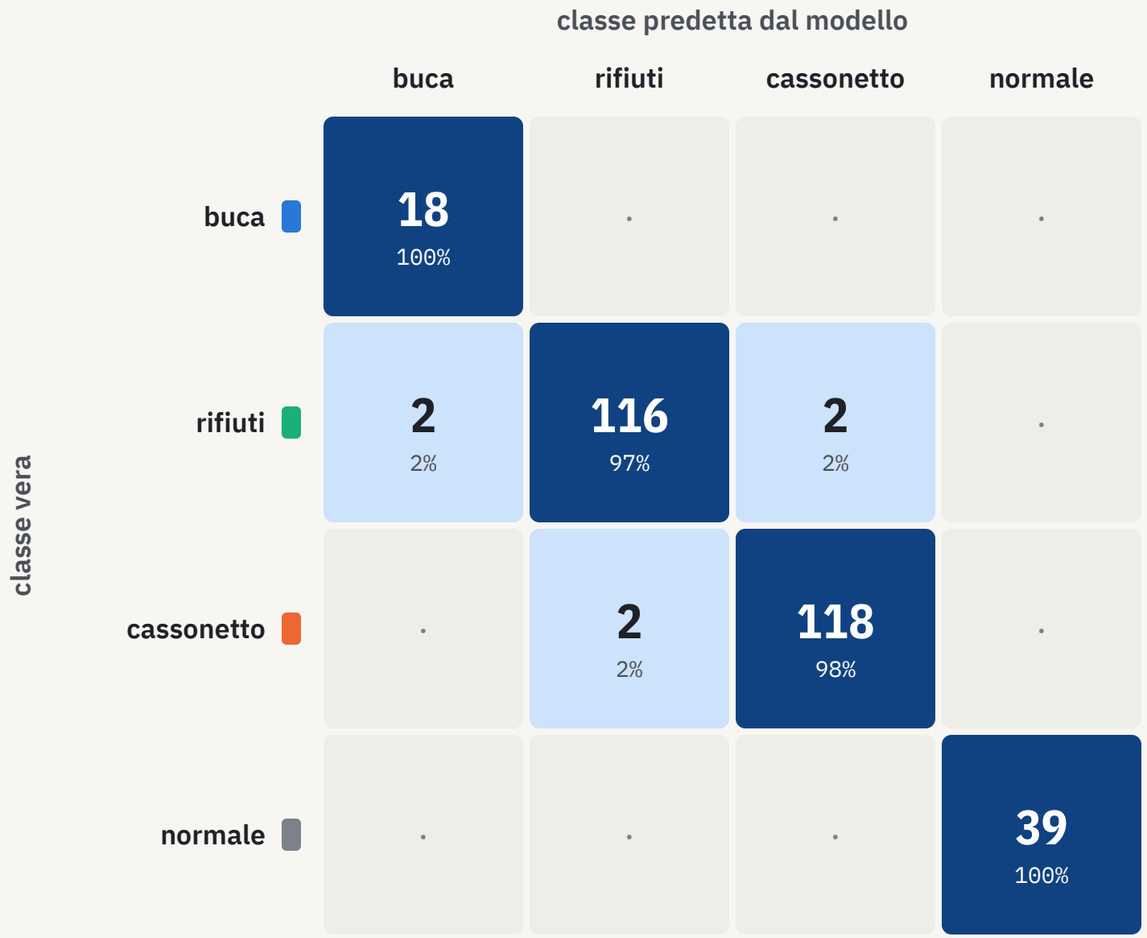


Le buche sono 6,7 volte meno delle altre classi

In addestramento ogni buca pesa 2,27 contro 0,34 di rifiuti e cassonetti, così il modello non impara a ignorarle.
Il rimedio vero è avere più foto di buche, meglio se riprese dal bus.

Dove sbaglia

Matrice di confusione sulle 297 foto di validation · modello int8 della board



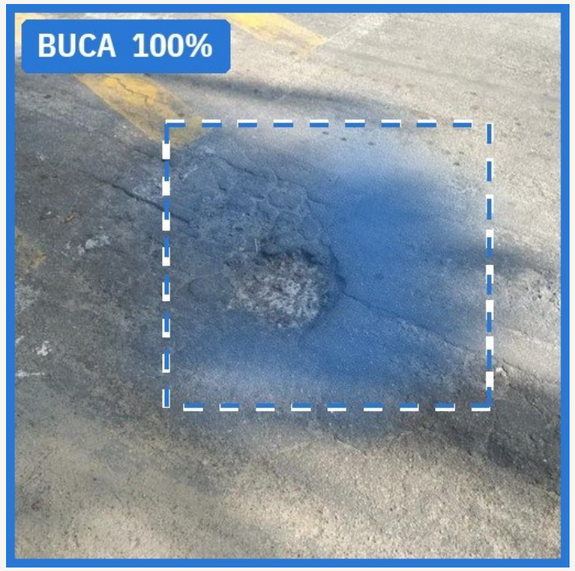
	precision	recall
buca	90,0%	100,0%
rifiuti	98,3%	96,7%
cassonetto	98,3%	98,3%
normale	100,0%	100,0%

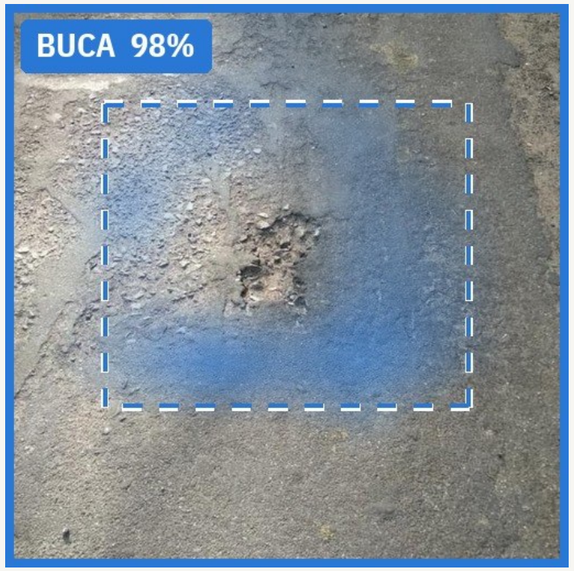
precision: quando dice X, quanto spesso è X
recall: delle foto X, quante ne trova

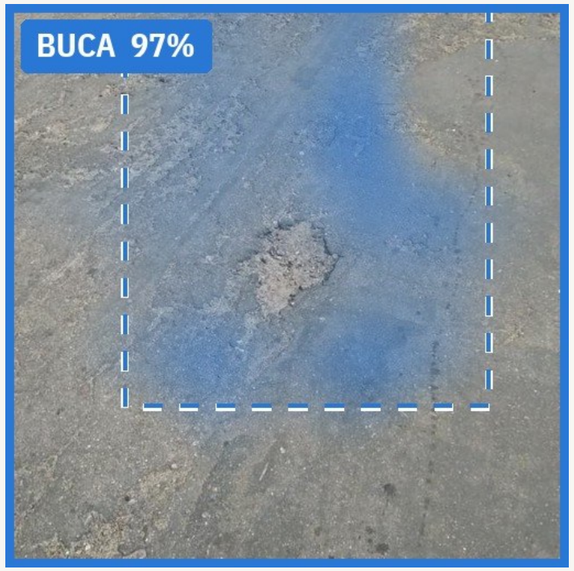
6 errori su 297, tutti fra rifiuti e cassonetto o rifiuti scambiati per buca. Nessuna foto di strada normale segnalata.

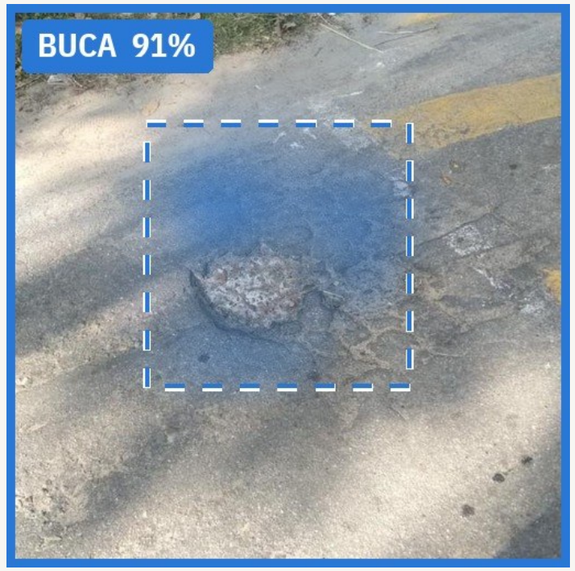
Buche riconosciute

La zona che ha deciso la risposta cade sulla buca







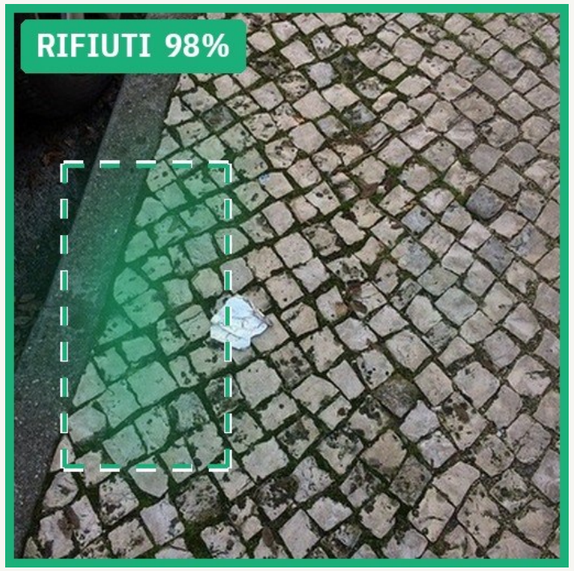


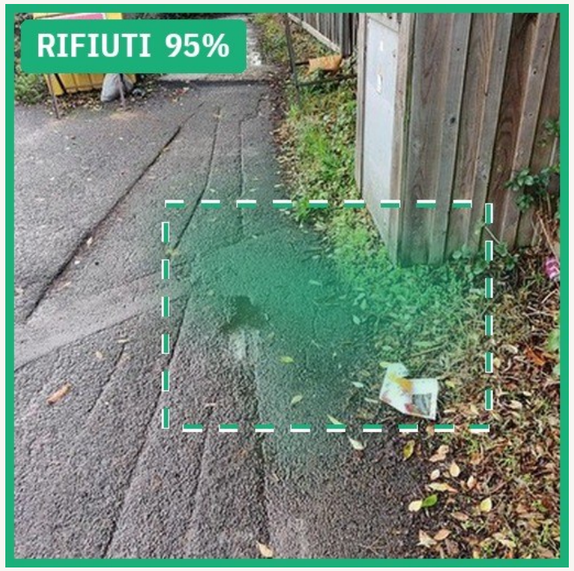
Etichetta e confidenza: modello int8 della UNO Q · zona colorata e riquadro: Grad-CAM, dove la rete ha guardato (non è un rilevamento)

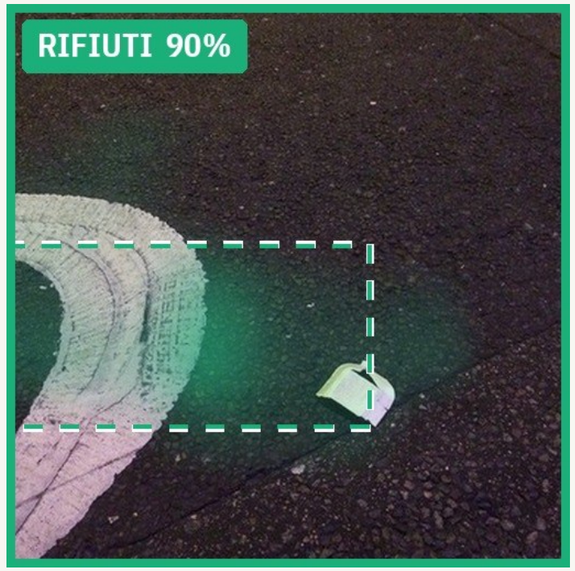
Rifiuti riconosciuti

La rete guarda l'oggetto e anche il terreno intorno









Etichetta e confidenza: modello int8 della UNO Q · zona colorata e riquadro: Grad-CAM, dove la rete ha guardato (non è un rilevamento)

Cassonetti pieni riconosciuti

Anche di notte, sfocati o in bianco e nero





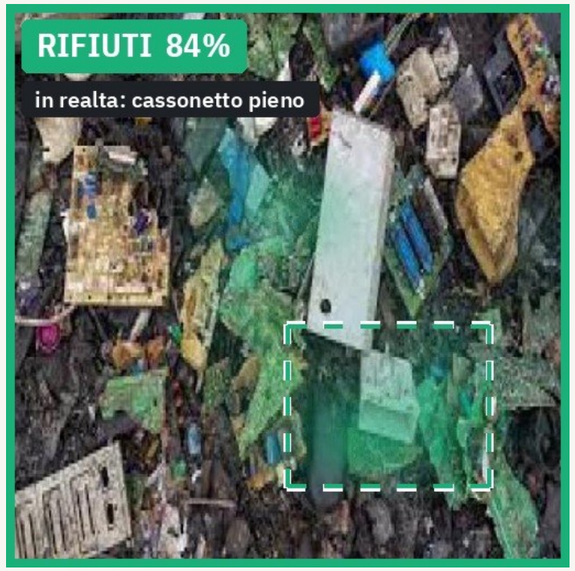




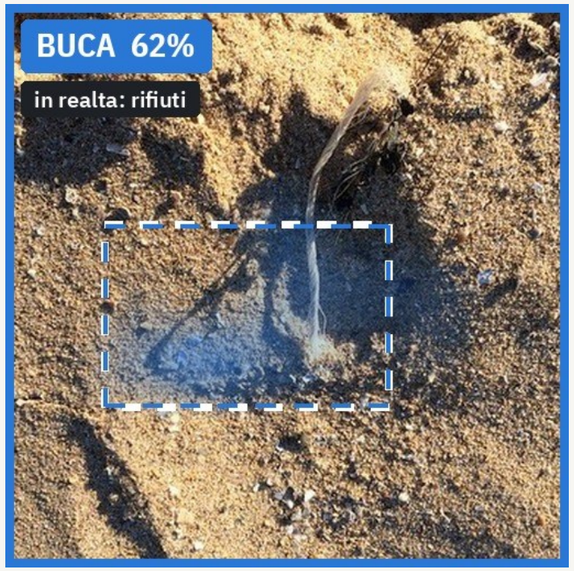
Foto di validation, mai viste in addestramento · Roboflow Universe e TACO, CC BY 4.0

Dove sbaglia

6 errori su 297 foto · 5 sotto la soglia 0,70, quindi sulla board scartati





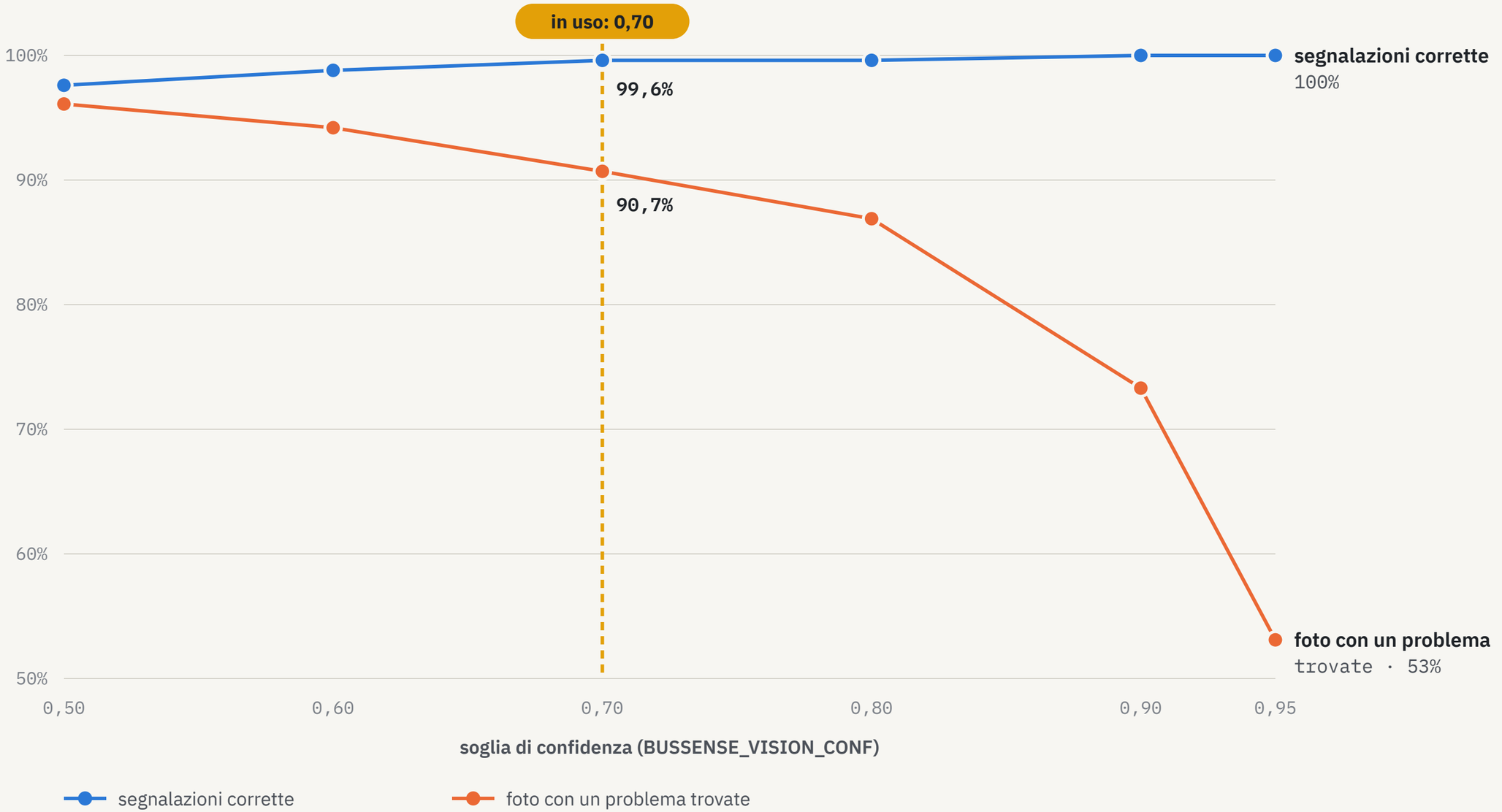




I primi due nel dataset sono "cassonetto pieno", ma nella foto ci sono cumuli di rifiuti e nessun cassonetto: l'errore potrebbe essere dell'etichetta.

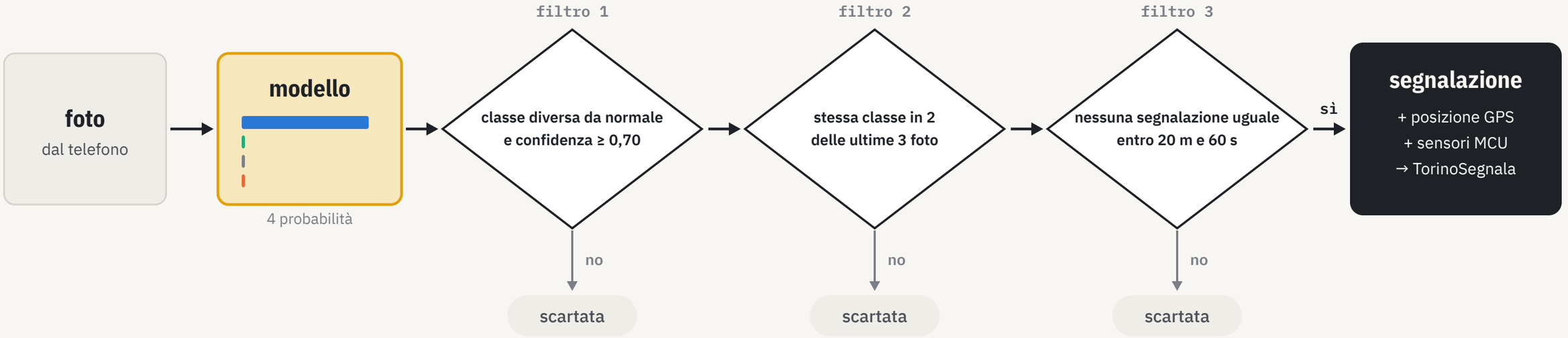
Quanto essere sicuri prima di segnalare

Al crescere della soglia si segnala meno ma si sbaglia meno · 297 foto di validation



Quando una foto diventa una segnalazione

Tre filtri in fila sulla UNO Q: una foto passa solo se li supera tutti



5 su 6

errori della validation fermati dalla soglia 0,70 (confidenza 50–65%)

19

foto corrette perse sotto soglia: il prezzo di pochi falsi allarmi

0

segnalazioni su 178 foto di una stanza, nella prova dal vivo

Perché serve la classe "normale"

Un classificatore sceglie sempre una delle classi che conosce. Senza "niente", inventa.

v1 · 3 classi

11

falsi allarmi in 7 minuti

telefono puntato su una stanza:
tavoli e zaini scambiati per
cassonetti e rifiuti, fino al 98%

v2 · + normale

0

segnalazioni su 178 foto

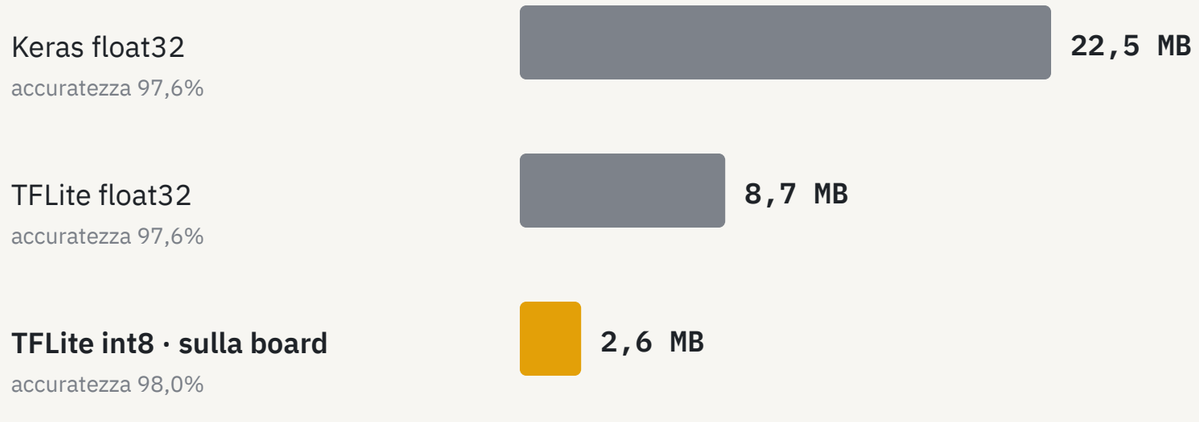
stessa stanza, stesso telefono:
195 frame di esempio bastano
a insegnare cos'è "niente"

Le 11 segnalazioni della v1 sono rimaste sulla board e non sono mai arrivate alla piattaforma.

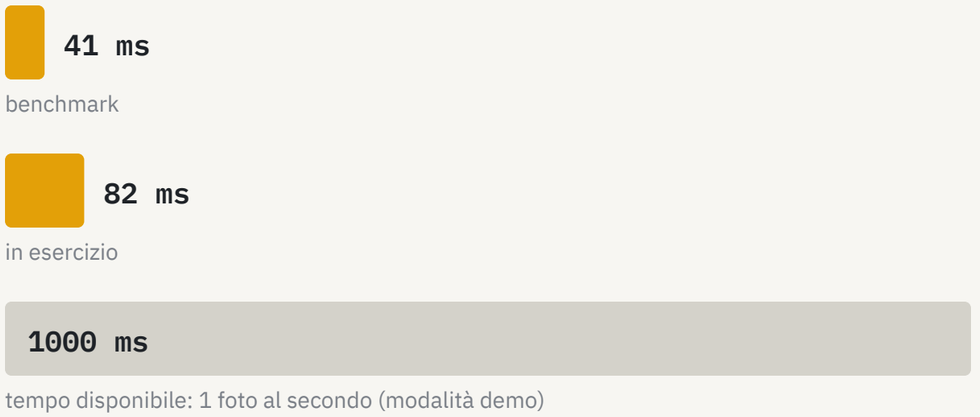
Piccolo e veloce abbastanza per la board

Stesso modello in tre formati, e quanto ci mette sulla UNO Q

Dimensione del file



Millisecondi per foto sulla UNO Q



−88% di spazio, accuratezza invariata

12 volte più veloce del necessario

4 core Cortex-A53
nessuna NPU usata: tutto in CPU

ai-edge-litert 2.3.0
pochi MB, niente TensorFlow

Python 3.13 · Debian 13
nel container dell'app App Lab

Coda da 1 foto
se resta indietro tiene l'ultima